

# Importing libraries

```
In [ ]: import nbconvert
from IPython.display import Image
import numpy as np
import pandas as pd
import seaborn as sns
import plotly.express as px
from matplotlib import pyplot as plt
from sklearn.model_selection import cross_val_score

from sklearn import metrics
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix, recall_score, precision_score, f1_score
from imblearn.over_sampling import RandomOverSampler
from imblearn.pipeline import Pipeline, make_pipeline
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import TomekLinks
from imblearn.combine import SMOTETomek
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import PrecisionRecallDisplay
```

## Import data

```
In [ ]: raw_df = pd.read_csv('card_fraud.csv')
```

- Data imbalance check

```
In [ ]: labels=["Genuine", "Fraud"]

fraud_or_not = raw_df["Class"].value_counts().tolist()
values = [fraud_or_not[0], fraud_or_not[1]]

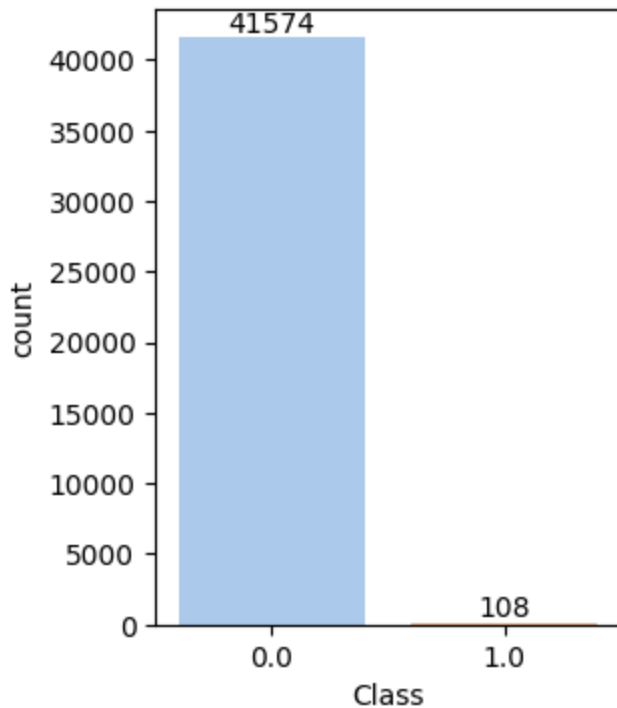
fig = px.pie(values=raw_df['Class'].value_counts(), names=labels, width=700, height=700)
fig.show()
```



```
In [ ]: plt.figure(figsize=(3,4))
ax = sns.countplot(x='Class',data=raw_df,palette="pastel")
for i in ax.containers:
    ax.bar_label(i,)
```

/tmp/ipython-input-386716584.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



```
In [ ]: print('Genuine:', round(raw_df['Class'].value_counts()[0]/len(raw_df) * 100,2), '%')
        print('Frauds:', round(raw_df['Class'].value_counts()[1]/len(raw_df) * 100,2), '%')
```

Genuine: 99.74 % of the dataset

Frauds: 0.26 % of the dataset

## Data pre-processing

- Dropping duplicated values

```
In [ ]: df = raw_df.copy()

df.drop_duplicates(inplace=True)
print("Duplicated values dropped succesfully")
print("'" * 100)
```

Duplicated values dropped succesfully

```
*****
*****
```

```
In [ ]: df = df.drop('Time', axis=1)
```

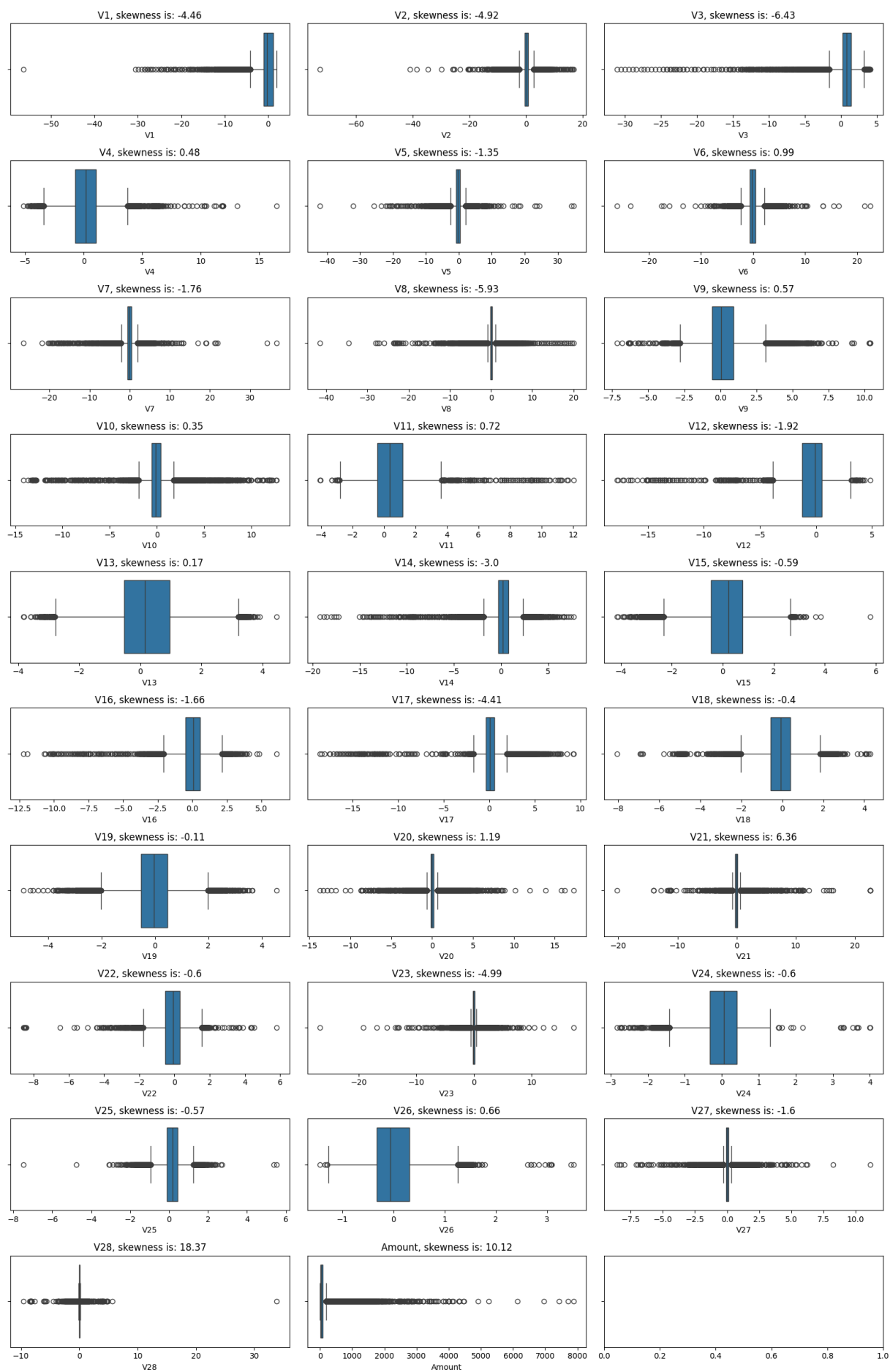
- Do not delete outliers

```
In [ ]: numeric_columns = (list(df.loc[:, 'V1':'Amount']))
```

```
In [ ]: # checking boxplots
def boxplots_custom(dataset, columns_list, rows, cols, subtitle):
    fig, axs = plt.subplots(rows, cols, sharey=True, figsize=(16,25))
```

```
fig.suptitle(suptitle,y=1, size=25)
axs = axs.flatten()
for i, data in enumerate(columns_list):
    sns.boxplot(data=dataset[data], orient='h', ax=axs[i])
    axs[i].set_title(data + ', skewness is: ' + str(round(dataset[data].skew(axis
boxplots_custom(dataset=df, columns_list=numeric_columns, rows=10, cols=3, suptitle
plt.tight_layout()
```

## Boxplots for each variable



```
In [ ]: def IQR_method (df,n,features):
        """
        Takes a dataframe and returns an index list corresponding to the observations
        containing more than n outliers according to the Tukey IQR method.
        """
        outlier_list = []

        for column in features:
            # 1st quartile (25%)
            Q1 = np.percentile(df[column], 25)
            # 3rd quartile (75%)
            Q3 = np.percentile(df[column],75)
            # Interquartile range (IQR)
            IQR = Q3 - Q1
            # outlier step
            outlier_step = 1.5 * IQR
            # Determining a list of indices of outliers
            outlier_list_column = df[(df[column] < Q1 - outlier_step) | (df[column] > Q
            # appending the list of outliers
            outlier_list.extend(outlier_list_column)

        # selecting observations containing more than x outliers
        outlier_list = Counter(outlier_list)
        multiple_outliers = list( k for k, v in outlier_list.items() if v > n )

        # Calculate the number of records below and above lower and above bound value r
        out1 = df[df[column] < Q1 - outlier_step]
        out2 = df[df[column] > Q3 + outlier_step]

        print('Total number of deleted outliers is:', out1.shape[0]+out2.shape[0])

        return multiple_outliers
```

```
In [ ]: # detecting outliers
        Outliers_IQR = IQR_method(df,1,numeric_columns)

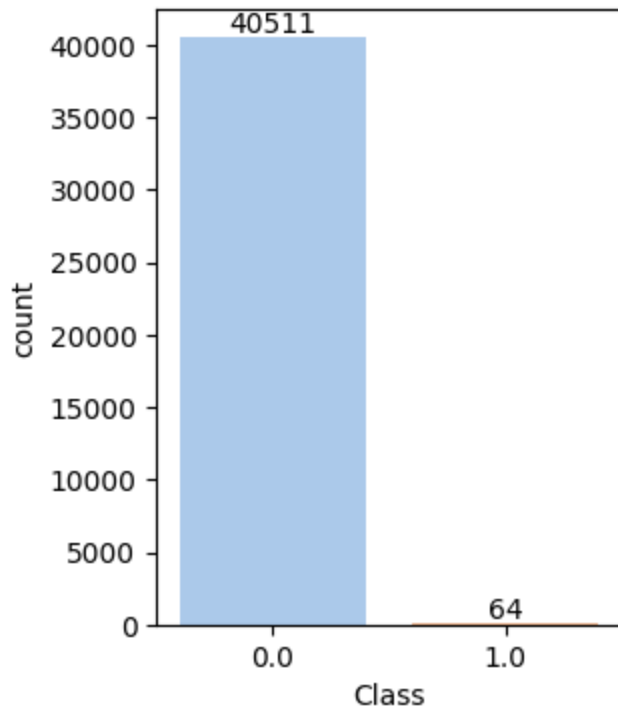
        # dropping outliers
        df_out = df.drop(Outliers_IQR, axis = 0).reset_index(drop=True)
```

Total number of deleted outliers is: 0

```
In [ ]: plt.figure(figsize=(3,4))
        ax = sns.countplot(x='Class',data=df_out,palette="pastel")
        for i in ax.containers:
            ax.bar_label(i,)
```

/tmp/ipython-input-2289713118.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



## Train-Test split

```
In [ ]: df = df.dropna()  
X = df.drop('Class', axis=1)  
y = df['Class']
```

```
In [ ]: X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size = 0.2)
```

## Feature scaling

```
In [ ]: # Creating function for scaling  
def StandardScaler (df, col_names):  
    features = df[col_names]  
    scaler = StandardScaler().fit(features.values)  
    features = scaler.transform(features.values)  
    df[col_names] = features  
  
    return df
```

```
In [ ]: col_names = ['Amount']  
X_train = StandardScaler (X_train, col_names)  
X_test = StandardScaler (X_test, col_names)
```

## Baseline

```
In [ ]: kf = StratifiedKFold(n_splits=5, shuffle=False)
```

```
In [ ]: rf = RandomForestClassifier(n_estimators=100, random_state=13)
```

- Optimizing for recall

```
In [ ]: score = cross_val_score(rf, X_train, y_train, cv=kf, scoring='recall')
print("Cross Validation Recall scores are: {}".format(score))
print("Average Cross Validation Recall score: {}".format(score.mean()))
```

```
Cross Validation Recall scores are: [0.93333333 0.66666667 0.66666667 0.875 0.875]
Average Cross Validation Recall score: 0.7883333333333333
```

- Hyperparameter tuning using GridSearchCV

```
In [ ]: params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [4, 6, 10, 12],
    'random_state': [13]
}

grid_rf = GridSearchCV(rf, param_grid=params, cv=kf,
                       scoring='recall').fit(X_train, y_train)
print('Best parameters:', grid_rf.best_params_)
print('Best score:', grid_rf.best_score_)
```

```
Best parameters: {'max_depth': 12, 'n_estimators': 50, 'random_state': 13}
Best score: 0.8283333333333334
```

```
In [ ]: y_pred = grid_rf.predict(X_test)
```

```
In [ ]: cm = confusion_matrix(y_test, y_pred)

rf_Recall = recall_score(y_test, y_pred)
rf_Precision = precision_score(y_test, y_pred)
rf_f1 = f1_score(y_test, y_pred)
rf_accuracy = accuracy_score(y_test, y_pred)

print(cm)
```

```
[[12418    1]
 [     6    26]]
```

```
In [ ]: ndf = [(rf_Recall, rf_Precision, rf_f1, rf_accuracy)]

rf_score = pd.DataFrame(data = ndf, columns=['Recall', 'Precision', 'F1 Score', 'Accuracy'])
rf_score.insert(0, 'Random Forest with', 'No Under/Oversampling')
rf_score
```



| Out[ ]: | Random Forest with    | Recall | Precision | F1 Score | Accuracy |
|---------|-----------------------|--------|-----------|----------|----------|
| 0       | No Under/Oversampling | 0.8125 | 0.962963  | 0.881356 | 0.999438 |

## Random resampling

- Random oversampling

```
In [ ]: ros = RandomOverSampler(random_state=42)
# fit and apply the transform
X_over, y_over = ros.fit_resample(X_train, y_train)
print('Genuine:', y_over.value_counts()[0], '/', round(y_over.value_counts()[0]/len(y_over), 2))
print('Frauds:', y_over.value_counts()[1], '/', round(y_over.value_counts()[1]/len(y_over), 2))
```

Genuine: 28974 / 50.0 % of the dataset  
Frauds: 28974 / 50.0 % of the dataset

```
In [ ]: random_overs_pipeline = make_pipeline(RandomOverSampler(random_state=42),
                                              RandomForestClassifier(n_estimators=100, random_state=42))
```

```
In [ ]: score2 = cross_val_score(random_overs_pipeline, X_train, y_train, scoring='recall', cv=5)
print("Cross Validation Recall Scores are: {}".format(score2))
print("Average Cross Validation Recall score: {}".format(score2.mean()))
```

Cross Validation Recall Scores are: [0.86666667 0.66666667 0.86666667 0.9375 0.66666667]  
Average Cross Validation Recall score: 0.7875

```
In [ ]: new_params = {'randomforestclassifier__' + key: params[key] for key in params}
grid_over_rf = GridSearchCV(random_overs_pipeline, param_grid=new_params, cv=kf, scoring='recall',
                           return_train_score=True)
grid_over_rf.fit(X_train, y_train)
print('Best parameters:', grid_over_rf.best_params_)
print('Best score:', grid_over_rf.best_score_)
```

Best parameters: {'randomforestclassifier\_\_max\_depth': 4, 'randomforestclassifier\_\_n\_estimators': 50, 'randomforestclassifier\_\_random\_state': 13}  
Best score: 0.8541666666666666

```
In [ ]: y_pred = grid_over_rf.best_estimator_.named_steps['randomforestclassifier'].predict(X_test)
```

```
In [ ]: cm = confusion_matrix(y_test, y_pred)

over_rf_Recall = recall_score(y_test, y_pred)
over_rf_Precision = precision_score(y_test, y_pred)
over_rf_f1 = f1_score(y_test, y_pred)
over_rf_accuracy = accuracy_score(y_test, y_pred)

print(cm)
```

```
[[12400  19]
 [    2  30]]
```

```
In [ ]: ndf = [(over_rf_Recall, over_rf_Precision, over_rf_f1, over_rf_accuracy)]

over_rf_score = pd.DataFrame(data = ndf, columns=['Recall','Precision','F1 Score',
over_rf_score.insert(0, 'Random Forest with', 'Random Oversampling')
over_rf_score
```

```
Out[ ]:      Random Forest with  Recall  Precision  F1 Score  Accuracy
0  Random Oversampling  0.9375   0.612245   0.740741   0.998313
```

- Random undersampling

```
In [ ]: rus = RandomUnderSampler(random_state=42)
# fit and apply the transform
X_under, y_under = rus.fit_resample(X_train, y_train)
print('Genuine:', y_under.value_counts()[0], '/', round(y_under.value_counts()[0]/1
print('Frauds:', y_under.value_counts()[1], '/', round(y_under.value_counts()[1]/len
```

Genuine: 76 / 50.0 % of the dataset  
Frauds: 76 / 50.0 % of the dataset

## SMOTE (Synthetic Minority Oversampling Technique)

```
In [ ]: smote_pipeline = make_pipeline(SMOTE(random_state=42),
RandomForestClassifier(n_estimators=100, random_state
score3 = cross_val_score(smote_pipeline, X_train, y_train, scoring='recall', cv=kf)
print("Cross Validation Recall Scores are: {}".format(score3))
print("Average Cross Validation Recall score: {}".format(score3.mean()))
```

Cross Validation Recall Scores are: [0.93333333 0.6 0.73333333 0.93333333 0.9375 ]  
Average Cross Validation Recall score: 0.8275

- GridSearchCV

```
In [ ]: new_params = {'randomforestclassifier__' + key: params[key] for key in params}
smote_rf = GridSearchCV(smote_pipeline, param_grid=new_params, cv=kf, scoring='reca
return_train_score=True)
smote_rf.fit(X_train, y_train)
print('Best parameters:', smote_rf.best_params_)
print('Best score:', smote_rf.best_score_)
```

Best parameters: {'randomforestclassifier\_\_max\_depth': 4, 'randomforestclassifier\_\_n\_estimators': 50, 'randomforestclassifier\_\_random\_state': 13}  
Best score: 0.8941666666666667

- Confusion matrix and scores

```
In [ ]: y_pred = smote_rf.best_estimator_.named_steps['randomforestclassifier'].predict(X_t
```

```
In [ ]: cm = confusion_matrix(y_test, y_pred)

smote_rf_Recall = recall_score(y_test, y_pred)
smote_rf_Precision = precision_score(y_test, y_pred)
smote_rf_f1 = f1_score(y_test, y_pred)
smote_rf_accuracy = accuracy_score(y_test, y_pred)

print(cm)

[[12388   31]
 [    0   32]]
```

```
In [ ]: ndf = [(smote_rf_Recall, smote_rf_Precision, smote_rf_f1, smote_rf_accuracy)]

smote_rf_score = pd.DataFrame(data = ndf, columns=['Recall', 'Precision', 'F1 Score',
smote_rf_score.insert(0, 'Random Forest with', 'SMOTE Oversampling')
smote_rf_score
```

```
Out[ ]:
```

|   | Random Forest with | Recall | Precision | F1 Score | Accuracy |
|---|--------------------|--------|-----------|----------|----------|
| 0 | SMOTE Oversampling | 1.0    | 0.507937  | 0.673684 | 0.99751  |

## Undersampling using Tomek Links

```
In [ ]: # define the undersampling method
#tomekU = TomekLinks(sampling_strategy='auto', n_jobs=-1)
tomekU = TomekLinks()

# fit and apply the transform
X_underT, y_underT = tomekU.fit_resample(X_train, y_train)
```

```
In [ ]: print('Genuine:', y_underT.value_counts()[0], '/', round(y_underT.value_counts()[0]
print('Frauds:', y_underT.value_counts()[1], '/', round(y_underT.value_counts()[1]/1
```

Genuine: 28972 / 99.74 % of the dataset  
Frauds: 76 / 0.26 % of the dataset

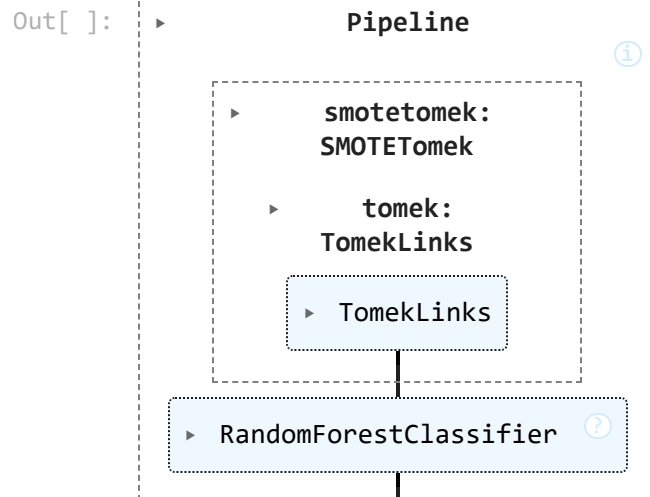
## Combining SMOTE and Tomek Links

- Training model

```
In [ ]: SMOTETomek_pipeline = make_pipeline(SMOTETomek(tomek=TomekLinks(sampling_strategy='
RandomForestClassifier(n_estimators=100, random_state
#cross_val_score(smote_pipeline, X_train, y_train, scoring='recall', cv=kf)
```

- GridSearchCV

```
In [ ]: SMOTETomek_rf = SMOTETomek_pipeline
SMOTETomek_rf.fit(X_train, y_train)
```



- Confusion matrix and scores

```
In [ ]: y_pred = SMOTETomek_rf.predict(X_test)
```

```
In [ ]: cm = confusion_matrix(y_test, y_pred)

SMOTETomek_rf_Recall = recall_score(y_test, y_pred)
SMOTETomek_rf_Precision = precision_score(y_test, y_pred)
SMOTETomek_rf_f1 = f1_score(y_test, y_pred)
SMOTETomek_rf_accuracy = accuracy_score(y_test, y_pred)

print(cm)
```

```
[[12415    4]
 [    6   26]]
```

```
In [ ]: ndf = [(SMOTETomek_rf_Recall, SMOTETomek_rf_Precision, SMOTETomek_rf_f1, SMOTETomek_rf_accuracy)]

SMOTETomek_rf_score = pd.DataFrame(data = ndf, columns=['Recall', 'Precision', 'F1 Score', 'Accuracy'])
SMOTETomek_rf_score.insert(0, 'Random Forest with', 'SMOTE + Tomek')
SMOTETomek_rf_score
```

```
Out[ ]:
```

|   | Random Forest with | Recall | Precision | F1 Score | Accuracy |
|---|--------------------|--------|-----------|----------|----------|
| 0 | SMOTE + Tomek      | 0.8125 | 0.866667  | 0.83871  | 0.999197 |

## Class weights in the models

```
In [ ]: rfb = RandomForestClassifier(n_estimators=100, random_state=13, class_weight="balanced")
```

```
In [ ]: score5 = cross_val_score(rfb, X_train, y_train, cv=kf, scoring='recall')
print("Cross Validation Recall scores are: {}".format(score5))
print("Average Cross Validation Recall score: {}".format(score5.mean()))
```

Cross Validation Recall scores are: [0.86666667 0.6 0.66666667 0.8 0.9375 ]  
Average Cross Validation Recall score: 0.7741666666666667

- GridSearchCV

```
In [ ]: grid_rfb = GridSearchCV(rfb, param_grid=params, cv=kf,
                               scoring='recall').fit(X_train, y_train)
```

- Confusion matrix and scores

```
In [ ]: y_pred = grid_rfb.predict(X_test)
```

```
In [ ]: cm = confusion_matrix(y_test, y_pred)

grid_rfb_Recall = recall_score(y_test, y_pred)
grid_rfb_Precision = precision_score(y_test, y_pred)
grid_rfb_f1 = f1_score(y_test, y_pred)
grid_rfb_accuracy = accuracy_score(y_test, y_pred)

print(cm)
```

```
[[12407  12]
 [    2  30]]
```

```
In [ ]: ndf = [(grid_rfb_Recall, grid_rfb_Precision, grid_rfb_f1, grid_rfb_accuracy)]

grid_rfb_score = pd.DataFrame(data = ndf, columns=['Recall', 'Precision', 'F1 Score',
grid_rfb_score.insert(0, 'Random Forest with', 'Class weights')
grid_rfb_score
```

```
Out[ ]:   Random Forest with  Recall  Precision  F1 Score  Accuracy
0          Class weights  0.9375   0.714286  0.810811  0.998876
```

## Performance comparison

```
In [ ]: predictions = pd.concat([rf_score, over_rf_score, smote_rf_score, SMOTETomek_rf_sco
predictions.sort_values(by=['Recall'], ascending=False)
```

| Out[ ]: | Random Forest with    | Recall | Precision | F1 Score | Accuracy |
|---------|-----------------------|--------|-----------|----------|----------|
| 2       | SMOTE Oversampling    | 1.0000 | 0.507937  | 0.673684 | 0.997510 |
| 4       | Class weights         | 0.9375 | 0.714286  | 0.810811 | 0.998876 |
| 1       | Random Oversampling   | 0.9375 | 0.612245  | 0.740741 | 0.998313 |
| 0       | No Under/Oversampling | 0.8125 | 0.962963  | 0.881356 | 0.999438 |
| 3       | SMOTE + Tomek         | 0.8125 | 0.866667  | 0.838710 | 0.999197 |

## Other metrics

- AOC-ROC Curve

```
In [ ]: ROCAUCscore = roc_auc_score(y_test, y_pred)
print(f"AUC-ROC Curve for Random Forest with Class weights: {ROCAUCscore:.4f}")
```

AUC-ROC Curve for Random Forest with Class weights: 0.9683

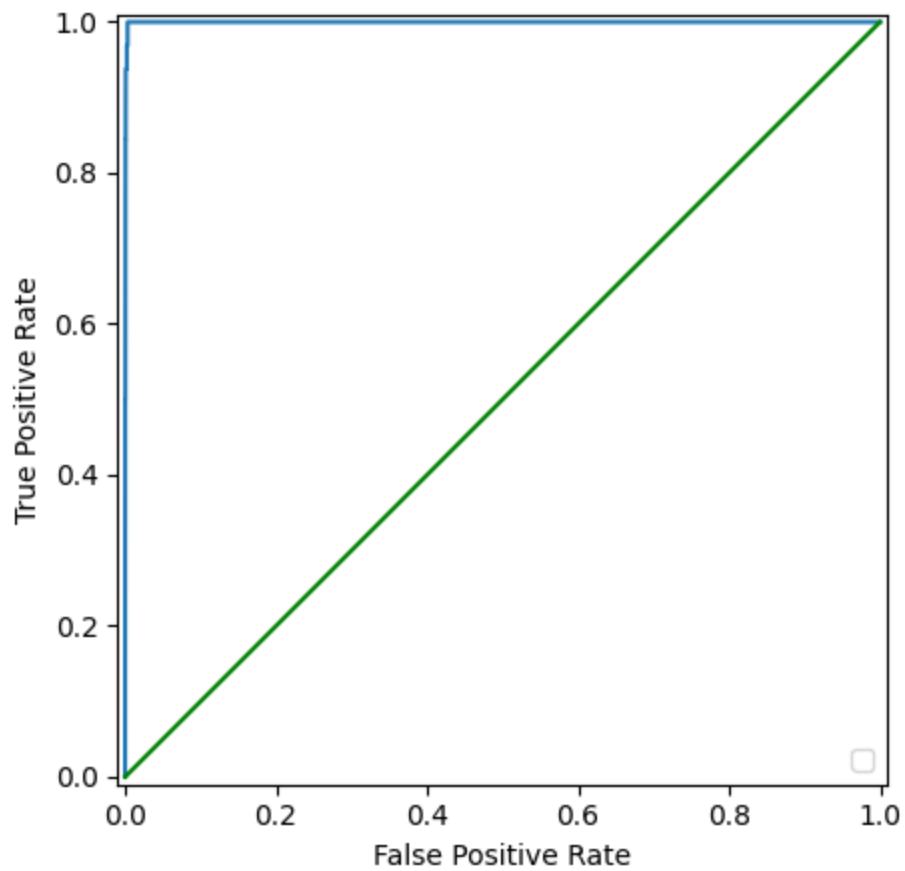
```
In [ ]: y_proba = grid_rfb.predict_proba(X_test)

from sklearn.metrics import roc_curve
from sklearn.metrics import RocCurveDisplay
def plot_auc_roc_curve(y_test, y_pred):
    fpr, tpr, _ = roc_curve(y_test, y_pred)
    roc_display = RocCurveDisplay(fpr=fpr, tpr=tpr).plot()
    roc_display.figure_.set_size_inches(5,5)
    plt.plot([0, 1], [0, 1], color = 'g')

plot_auc_roc_curve(y_test, y_proba[:, 1])
```

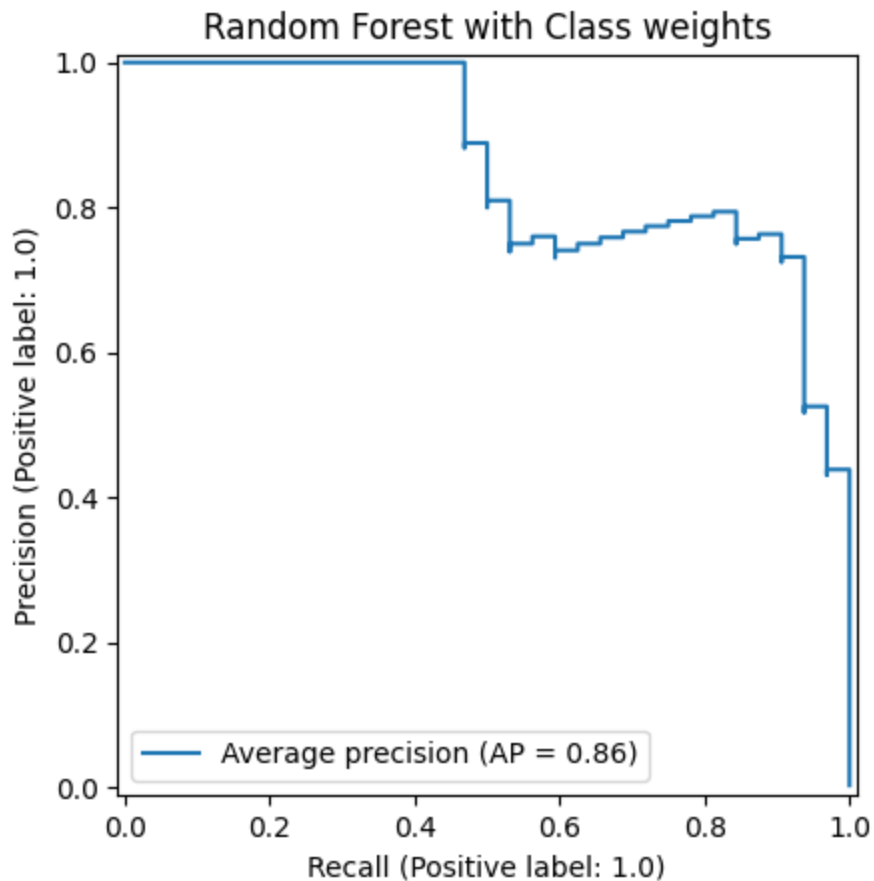
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/\_plot/roc\_curve.py:189: User Warning:

No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.



- Precision-Recall Curve

```
In [ ]: display = PrecisionRecallDisplay.from_estimator(  
    grid_rfb, X_test, y_test, name="Average precision")  
_ = display.ax_.set_title("Random Forest with Class weights")
```



## Documentation

---

### Summary

In this project we are classifying card transactions as being fraudulent or not.

For this we are using an imbalanced dataset, as most card transactions are not fraudulent.

To combat this issue, we are trying different approaches to balance the dataset to avoid bias. All of them will be used in a Random Forest model, as it is the most accurate on average.

---

### Implementation

First thing was to check the imbalance of the dataset. We have a percentage of 99.7% genuine transactions which leaves only a small amount of fraudulent transactions to work with.

Next step is to pre-process the data. This usually means dropping duplicates, entries with no value and outliers. In our case, we have to keep the outliers as removing them would only leave 64 fraudulent transactions.



After splitting the dataset into train and test subsets, we start implementing the different strategies to fix the imbalance. We'll be using GridSearchCV for hyperparameter tuning together with Random Forest.

First approach was the Random Resampling, which can be Random Oversampling or Random Undersampling, although in our case the undersampling method is not applicable as it would only leave 76 genuine transactions and 76 fraudulent ones.

Next approach was the SMOTE, followed by Tomek Links. SMOTE is an oversampling technique, while Tomek Links is an undersampling one. Those two can be combined into a single technique called SMOTETomek.

Lastly, another approach is class weights, meaning the minority class can have a higher weight to combat the imbalance.

---

## Results

The recall, precision, F1 scores and accuracies for the used techniques are the following:

|   | Random Forest with    | Recall | Precision | F1 Score | Accuracy |
|---|-----------------------|--------|-----------|----------|----------|
| 2 | SMOTE Oversampling    | 1.0000 | 0.507937  | 0.673684 | 0.997510 |
| 4 | Class weights         | 0.9375 | 0.714286  | 0.810811 | 0.998876 |
| 1 | Random Oversampling   | 0.9375 | 0.612245  | 0.740741 | 0.998313 |
| 0 | No Under/Oversampling | 0.8125 | 0.962963  | 0.881356 | 0.999438 |
| 3 | SMOTE + Tomek         | 0.8125 | 0.866667  | 0.838710 | 0.999197 |

The SMOTE Oversampling achieved best recall score, but at the cost of the lowest precision of 0.5 and lowest F1 score of 0.67.

The reference model without under/oversampling achieved 0.81 recall.

The most advanced technique, SMOTETomek, was the worst at recall.